

Tổng Hợp Kiến Thức

1. Mô hình MVC là gì?

- **MVC (Model-View-Controller):** Mẫu kiến trúc tách ứng dụng thành 3 phần:
 - **Model:**
 - **Trách nhiệm:** Quản lý dữ liệu, logic nghiệp vụ, tương tác Database (DB).
 - **Ví dụ:** Sequelize Models (`Category` , `Product` , etc.) - định nghĩa cấu trúc bảng, cung cấp hàm CRUD (`findAll` , `create` , `findByPk`).
 - **View (Trong API):**
 - **Trách nhiệm:** Trình bày dữ liệu cho client.
 - **Ví dụ:** **JSON Response** được gửi về từ API.
 - **Controller:**
 - **Trách nhiệm:** Trung gian xử lý request, gọi Model lấy/cập nhật dữ liệu, chuẩn bị và gửi View (JSON response).
 - **Ví dụ:** Logic trong các hàm `async (req, res, next)` sẽ được chuyển vào đây.
- **Luồng API cơ bản:** `Client Request` → `Route` → `Middleware(s)` → `Controller` → `Model` → `Controller` → `View (JSON Response)` → `Client` .

2. Lợi ích của MVC

- **Tách biệt Trách nhiệm:** Code rõ ràng, dễ hiểu hơn.
- **Dễ Bảo trì & Mở rộng:** Sửa đổi ít ảnh hưởng đến các phần khác.
- **Tái sử dụng:** Logic Model dùng lại được.
- **Dễ Kiểm thử (Test):** Test từng phần riêng biệt.
- **Dễ Hợp tác:** Nhóm làm việc song song hiệu quả hơn.

3. Cấu trúc Thư mục MVC Đề xuất

```
your-project-root/
├── config/
├── controllers/      # ← CHỨA LOGIC XỬ LÝ REQUEST
│   ├── categoryController.js
│   └── productController.js
├── middlewares/      # (errorHandler, requestLogger, validationErrorHandler)
├── models/           # ← CHỨA ĐỊNH NGHĨA DỮ LIỆU (SEQUELIZE)
│   ├── category.js
│   ├── product.js
│   └── index.js
├── node_modules/
├── routes/           # ← CHỨA ĐỊNH NGHĨA ENDPOINT & GẮN CONTROLLER
│   ├── categoryRoutes.js
│   └── productRoutes.js
├── validators/       # ← CHỨA QUY TẮC VALIDATION (express-validator)
│   ├── categoryValidator.js
│   └── productValidator.js
├── .env
├── .gitignore
├── package.json
└── server.js         # (Khởi tạo app, gán routes, middleware)
```

4. Code Snippet Mẫu

Controller (controllers/someController.js)

```
const { SomeModel } = require('../models'); // 1. Import Model

// 2. Export các hàm xử lý
exports.getResource = async (req, res, next) => {
  try {
    // 3. Tương tác với Model
    const data = await SomeModel.findAll();
    // 4. Gửi Response (View JSON)
    res.status(200).json(data);
  } catch (error) {
    // 5. Xử lý lỗi
    next(error);
  }
};

exports.createResource = async (req, res, next) => {
  // Input đã được validate bởi middleware
  try {
    const { field1, field2 } = req.body;
    const newData = await SomeModel.create({ field1, field2 });
    res.status(201).json(newData);
  } catch (error) {
    next(error);
  }
}
```

Route (routes/someRoutes.js)

```
const express = require('express');
const router = express.Router();
const someController = require('../controllers/someController'); // 1. Import Controller
const { validateCreate } = require('../validators/someValidator'); // 2. Import Validators
const handleValidationErrors = require('../middlewares/validationErrorHandler'); // 3. Import Middleware

// 4. Định nghĩa route và gắn middleware + controller
router.get('/', someController.getResource);
router.post('/',
  validateCreate(), // Middleware 1: Quy tắc validation
  handleValidationErrors, // Middleware 2: Xử lý lỗi validation
  someController.createResource // Hàm Controller cuối cùng
);

module.exports = router;
```